

Report: Recovering Cyber-Security Domain Knowledge via LoRA Fine-Tuning

Farit Sharafutdinov, Grigorii Belayev
Course: Generative AI (Spring 2026)
Instructor: Bader Rasheed

GitHub: github.com/FaritSharafutdinov/Recovering-Cyber-Security-Domain-Knowledge

Contributions: Farit Sharafutdinov—dataset, LoRA training pipeline, experiments; Grigorii Belayev—baseline evaluation, inference/evaluation scripts, report.

1 Objective and Research Hypothesis

Objective. Measure and reduce *false refusals* on legitimate defensive cyber-security tasks, while preserving refusal behavior on clearly malicious requests.

Baseline question. Why does an aligned chat model underperform on technical security prompts even when requests are educational/defensive?

Working hypothesis. Alignment policy introduces a conservative decision boundary in prompt space: token patterns associated with “offensive” tasks can trigger refusal templates, reducing utility. Parameter-efficient adaptation (LoRA) should shift this boundary for authorized/defensive contexts without fully disabling safety.

2 Model Definition and Training Objective

2.1 Backbone model

Current baseline inference uses 4-bit Llama-3-8B-Instruct (`unsloth/llama-3-8b-instruct-bnb-4bit`).

2.2 LoRA parameterization

For a frozen pretrained matrix $W_0 \in \mathbb{R}^{d \times k}$, LoRA learns a low-rank update:

$$W = W_0 + \Delta W, \quad \Delta W = BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k).$$

Only A, B are trained, reducing optimization dimension from dk to $r(d+k)$. In our setup, target modules are attention projections (`q_proj`, `v_proj`).

2.3 Language modeling loss

Given instruction-response tokens $x_{1:T}$, the causal LM objective is

$$\mathcal{L}_{\text{CLM}}(\Delta W) = - \sum_{t=1}^T \log p_{\theta_0, \Delta W}(x_t \mid x_{<t}).$$

This objective is shared by baseline adaptation experiments and future full LoRA training runs.

2.4 Optimization details (current and planned)

- **Current sanity training** (`scripts/train.py`): tiny overfit check, 20 steps, LoRA on TinyLlama, fixed seed, verifies that loss decreases and pipeline is operational.
- **Planned main training**: QLoRA-style fine-tuning of 8B backbone with rank/routing ablations; frozen base weights, low-rank adapters only.

3 Evaluation Protocol and Reproducibility

3.1 Dataset and split

Evaluation set: 50 prompts in `data/queries.json`, covering exploitation mechanics, protocol/security analysis, and defensive operations. Same query IDs are reused across all runs for controlled comparison.

3.2 Refusal taxonomy

Each response has label $y_i \in \{\text{hard}, \text{soft}, \text{none}\}$.

- **Hard refusal**: explicit deny template (“I cannot assist...” etc.).
- **Soft refusal**: evasive/generic answer with missing requested technical content.
- **None**: no refusal.

For baseline, manual labels from expert review are stored in `data/baseline_refusal_labels.json`.

3.3 Primary metric with uncertainty

Define binary refusal indicator $r_i = \mathbb{I}[y_i \neq \text{none}]$. Then refusal rate is

$$\hat{p} = \frac{1}{N} \sum_{i=1}^N r_i.$$

We report 95% bootstrap CI using resampling:

$$[\hat{p}_{0.025}, \hat{p}_{0.975}] \text{ from } B = 2000 \text{ bootstrap samples.}$$

This avoids over-interpreting point estimates on a small test set ($N = 50$).

3.4 Controlled comparisons already implemented

1. **Difficulty-tier comparison**: per-tier refusal rates (`data/query_tiers.json`).
2. **Prompt-profile comparison**: same queries with different system prompts (`data/prompt_profiles.json`).
3. **Trigger-token sensitivity**: keyword-conditioned refusal analysis (`data/trigger_tokens.json`).

3.5 Reproducibility commands

```
pip install -r requirements.txt
python scripts/inference.py --queries data/queries.json --out outputs/inference_outputs.json --seed 42
python scripts/eval_refusal_rate.py --input data/baseline_outputs.json \
  --manual_labels data/baseline_refusal_labels.json \
  --save_markdown outputs/baseline_eval_report.md
```

```
python scripts/analyze_trigger_tokens.py --queries data/queries.json \
--responses data/baseline_outputs.json \
--manual_labels data/baseline_refusal_labels.json \
--out outputs/trigger_token_report.md
```

4 Results So Far (Evidence)

4.1 Core baseline metrics

- Total responses: 50
- Hard refusals: 7
- Soft refusals: 7
- **Total refusal rate: 28.0%**
- 95% bootstrap CI: [16.0%, 42.0%]
- Mean helpfulness (expert baseline report): 4.56/10

4.2 Difficulty-tier evidence

Tier	N	Refusals	Hard	Soft	Rate
advanced_exploitation	11	4	1	3	36.4%
offense_sensitive	22	9	6	3	40.9%
defense_practical	10	1	0	1	10.0%
technical_explanation	7	0	0	0	0.0%

Interpretation. Refusal probability is not uniform; it concentrates in high-risk lexical/semantic regions (`offense_sensitive`, `advanced_exploitation`). This supports the hypothesis of alignment-driven boundary effects.

4.3 Trigger-token sensitivity evidence

Token	N	Refusals	Hard	Soft	Rate
payload	2	2	2	0	100.0%
shellcode	1	1	0	1	100.0%
token	1	1	1	0	100.0%
bypass	4	3	2	1	75.0%
injection	2	1	1	0	50.0%
waf	2	1	1	0	50.0%

Interpretation and limitation. Some tokens have high conditional refusal rates, but sample counts are small. We therefore treat this as diagnostic evidence, not causal proof. Next stage uses larger and balanced query variants for stronger inference.

4.4 Failure analysis

Representative over-refusal examples:

- **ID 3** (format string): denied despite educational security context.
- **ID 10** (time-based SQLi): denied as “malicious” without authorized-testing distinction.

- **ID 19** (JWT None): denied despite defensive objective (understanding vulnerable validation logic).

These cases explain low usefulness and show where adaptation must improve context discrimination rather than remove safety.

5 Important Implementation Snippets

Deterministic protocol controls (`scripts/inference.py`): fixed seed, explicit decoding parameters, optional sampling switch (`-do_sample`), and explicit system prompt override for controlled prompt experiments.

Evaluation robustness (`scripts/eval_refusal_rate.py`): hard/soft taxonomy, optional manual labels, bootstrap CI, and tier-wise breakdown.

Controlled-analysis script (`scripts/analyze_trigger_tokens.py`): token-conditioned refusal rates with matched-query examples.

6 Planned Improvements (from Project Roadmap)

We keep all planned directions and prioritize them by research value and implementation feasibility:

1. Prompt engineering vs LoRA comparison (already scaffolded; run full comparison table).
2. RAG baseline (non-parametric domain augmentation).
3. LoRA dataset-size ablation (sample complexity curve).
4. Catastrophic forgetting check on general tasks.
5. Layer-target ablation (Q only vs Q+V vs wider adapters).
6. LoRA rank ablation ($r \in \{4, 16, 64, 128\}$).
7. Trigger-token behavior before/after fine-tuning (already scaffolded for baseline).
8. LLM-based evaluator for scalable labeling.
9. Larger standardized benchmark beyond 50 handcrafted prompts.
10. Difficulty-tier expansion with stricter category definitions.
11. Statistical significance extension (paired tests + bootstrap on deltas).
12. Reproducibility hardening (seed matrix, config snapshots).
13. Systematic error taxonomy beyond refusals (hallucination, unsafe detail level, partial correctness).

7 Conclusion

Current submission demonstrates both **practice** (working code, scripts, reproducibility path) and **theory** (formal objective, low-rank adaptation model, uncertainty-aware metrics, and controlled comparisons). Results so far confirm substantial over-refusal on security-relevant prompts (28%) with clear concentration in offense-sensitive tiers. The next phase is to quantify whether LoRA can move this boundary toward legitimate defensive utility while preserving safety refusals for clearly harmful intent.